

RAP 後端開發練習 6：Validations (資料完整性檢查)

Lecture

這一課要解決什麼問題

rap02 的 `zrap02_status` / `zrap02_priority` 是靠 Domain 固定值清單限制輸入 (`O` / `D` 兩個合法值)，但 rap02 當時已經提醒過：**Domain 固定值清單只在 UI / 畫面層級生效，不是資料庫硬約束**——直接用 Open SQL 或繞過畫面檢查的方式，一樣可以塞進不合法的值。**Validation (驗證)** 就是 RAP 框架用來在應用層真正擋下非法資料的機制：存檔 (`on save`) 時檢查資料是否一致，不一致就拒絕整筆交易並回報錯誤訊息，比單純的 Domain 固定值清單更可靠。

延續 rap05 的分工：

- **Part A：Managed Validation 語法**——這系統上可以編譯、可以啟用，一樣沒辦法真正執行 (Managed Runtime 白名單限制)；語法模式跟 rap05 的 Determination 高度相似 (同樣要用 `obsolete` 語法)，但回應參數的細節有一個新發現的差異 (見下方)
- **Part B：Unmanaged 一樣沒有宣告式 Validation**——查證官方文件確認跟 Determination 同一個限制 (非 Draft 不支援)，等效邏輯寫在 `CREATE` 方法裡，已用 `programrun` 驗證成功，而且意外揭露了一個 **Managed / Unmanaged** 在「部分失敗時怎麼處理」上的行為差異 (見 Part C)
- **Part C：對照總表**

Part A：Managed Validation 語法 (知識儲備，這系統無法執行)

在看完整範例之前，先認識會用到的語法元素 (查證官方 ABAP 語言文件 `ABENBDL_VALIDATIONS` / `ABAPHANDLER_METH_VALIDATE` 確認)：

- **validation <名稱> on save { field <欄位> }**：寫在 BDEF 的 `{ }` 區塊裡，宣告一個 Validation。跟 **Determination** 不同：**Validation** 只能用 `on save`，沒有 `on modify` 這個選項 (官方文件的語法只列了 `on save` 一種)；觸發條件跟 Determination 一樣可以是 `create/update/delete` 或 `field <欄位>` (欄位被改到才觸發)。
- **Validation 失敗的後果跟 Determination 完全不同**——這是 Validation 存在的核心價值：
 - Determination：只是「算欄位的值」，本身不會讓交易失敗。
 - Validation：檢查資料是否合法，一旦有任何一筆實例驗證失敗，當次 **RAP 交易 (Transactional Buffer)** 裡的所有變更全部被拒絕 (不是只拒絕驗證失敗的那一筆)，官方文件用詞是「one inconsistency leads to a rejection of all the content in the transactional buffer」——這是 Managed Validation 一個很重要、容易被忽略的行為，Part C 會對照 Unmanaged 手寫版本的差異。

沿用 rap05 已經延伸過的 `ZI_RAP02_TASK` BDEF，這是加上 Validation 後的完整版本：

```
managed implementation in class zbp_i_rap02_task unique;

define behavior for ZI_RAP02_TASK alias Task
persistent table zrap02_task
etag created_at
```

```
lock master
{
    create;
    update;
    delete;

    field ( mandatory ) description;

    determination setCreationInfo on save { create; }
    validation validateStatus on save { field status; }
}
```

validateStatus 這個 Validation 要做的事：用應用層邏輯真正檢查 **status** 只能是 **O (Open)** 或 **D (Done)** ——呼應開頭提到的「Domain 固定值清單只在 UI 層生效」這個問題，Validation 才是資料庫層 / 應用層真正擋下非法值的機制。

Validation 的 Handler Method：跟 Determination 同一個 **obsolete** 語法家族，但有一個新差異

延續 rap05 已經確認的模式，Validation 的 Handler Method 一樣要用官方標成「**obsolete**」的舊式關鍵字（這次是 **FOR VALIDATION**，不是官方新式的 **FOR VALIDATE ON SAVE**）：

```
METHODS validateStatus FOR VALIDATION Task~validateStatus
IMPORTING keys FOR Task.
```

寫法規則跟 rap05 的 **FOR DETERMINATION** 完全一致（**FOR VALIDATION <alias>~<validation名稱>+ IMPORTING keys FOR <alias>**，不重複驗證名稱），這次沒有再逐步踩錯——因為 **rap05** 已經把這個語法家族的規則摸清楚了，直接套用就對了，這正是「花時間搞懂一個語法家族的規則，之後同家族的新成員可以直接套用」的具體例子。

⚠ 但方法本體有一個 **rap05** 沒遇過的新差異：**failed / reported** 的型別，這裡是直接可用的表格，不需要 **-task** 這種按實體別名分類的寫法（因為 **ZI_RAP02_TASK** 只有一個實體，沒有子實體，衍生型別直接簡化成單一表格）：

```
CLASS lhc_task DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
    METHODS setCreationInfo FOR DETERMINATION Task~setCreationInfo
        IMPORTING keys FOR Task.

    METHODS validateStatus FOR VALIDATION Task~validateStatus
        IMPORTING keys FOR Task.
ENDCLASS.

CLASS lhc_task IMPLEMENTATION.

METHOD setCreationInfo.
    " ... ( rap05 已教過，略 )
```

```

ENDMETHOD.

METHOD validateStatus.
  READ ENTITIES OF zi_rap02_task IN LOCAL MODE
  ENTITY task
    FIELDS ( status ) WITH CORRESPONDING #( keys )
  RESULT DATA(tasks).

  LOOP AT tasks INTO DATA(ls_task).
    IF ls_task-status <> 'O' AND ls_task-status <> 'D'.
      APPEND VALUE #( %key = ls_task-%key ) TO failed.
      APPEND VALUE #( %key = ls_task-%key
                      %msg = new_message_with_text(
                        severity = if_abap_behv_message=>severity-error
                        text      = 'Status must be O or D' ) )
      TO reported.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

- `new_message_with_text( severity = ... text = ... )`：建立一個 RAP 訊息物件最簡單的方式——`severity` 用 `if_abap_behv_message=>severity-error`（也有 `-warning` / `-information`）；`text` 直接給字串當訊息內容（實務上更常見的做法是用 T100 訊息類別 + `new_message( )` 帶入變數，但這種方式要另外建 Message Class，這裡先用最簡單的字面字串示範）。
- `failed` / `reported` 只要 `APPEND ... TO failed.` / `TO reported.`，不用（也不能）加 `-task`——這是這次跟 rap05 對照出來的新發現：同一個「**obsolete Handler Method 家族**」裡，**FOR DETERMINATION** / **FOR VALIDATION** 這兩種 **Handler**，`failed`/`reported` 在單一實體 **BDEF** 的情境下都是直接可用的表格；下面 Part B 會看到 **FOR MODIFY (create)** 這種 **Handler** 反而不是這樣，這個差異值得特別注意。

`ZBP_I_RAP02_TASK` 加了 `validateStatus` 之後，`checkruns / sap_inactive_objects` 確認語法正確、成功啟用——一樣不要嘗試執行，原因同 rap05（Managed Runtime 白名單限制，見 rap03 Part C）。

Part B：Unmanaged 一樣沒有宣告式 Validation——官方文件同一條限制

查證官方文件 `ABENBDL_VALIDATIONS` 原文，Validation 的可用範圍描述跟 rap05 查過的 Determination 一字不差是同一條規則：

Validations are available for:

- Managed RAP BOs
- Unmanaged **and draft-enabled** RAP BOs
- **Caution: Not available for unmanaged, non-draft RAP BOs.**

所以跟 rap05 一樣，`ZI_RAP03_UMTEST` (Unmanaged、沒有 Draft) 沒有 `validation ... on save { }` 這種宣告式語法可用，等效邏輯要直接寫在 `CREATE` 方法裡——用同一套 `failed / reported` 機制拒絕不合法的輸入，只是這次呼叫的時機是你自己決定 (`create` 方法一開始檢查)，不是框架在 `on save` 自動觸發。

這一課選用「`descr` (描述) 不能是空字串」當驗證規則 (比 Managed 那邊驗證 `status` 固定值更貼近 Unmanaged 這張簡單表格的欄位設計)：

```
METHOD create.  
  DATA(ls_info) = determine_creation_info( ).  
  
  LOOP AT it_create INTO DATA(ls_create).  
    IF ls_create-descr IS INITIAL.  
      APPEND VALUE #( %cid = ls_create-%cid ) TO failed-test.  
      APPEND VALUE #( %cid = ls_create-%cid  
                      %msg = new_message_with_text(  
                        severity = if_abap_behv_message=>severity-error  
                        text      = 'Description must not be empty' ) )  
        TO reported-test.  
      CONTINUE.  
    ENDIF.  
  
    INSERT zrap03_umtest FROM @( VALUE #(  
      client      = sy-mandt  
      id          = ls_create-id  
      descr       = ls_create-descr  
      created_at  = ls_info-created_at  
      created_by  = ls_info-created_by ) ).  
  ENDLLOOP.  
ENDMETHOD.
```

⚠ 這裡出現一個容易搞混、但很有意義的對照：跟上面 **Part A** 的 **Validation Handler** 相反，`create` 方法 (**FOR MODIFY Handler**) 的 `failed / reported` 這次要用 `-test` 這種按實體別名分類的寫法 (`failed-test`)——直接寫 `failed` (不加 `-test`) 會報 `"FAILED" is not an internal table.`。原因是：FOR MODIFY 這種 Handler 是綁定「CUD 操作」本身，即使 BDEF 只有一個實體，衍生型別依然固定用「按實體分類的結構」表示 (因為一個 Modify Handler 理論上有可能同時處理多種操作或關聯到其他實體)；FOR DETERMINATION / FOR VALIDATION 這種 Handler 是綁定到「某個特定 Determination / Validation 名稱」本身，單一實體時型別會直接簡化成表格。一句話記憶：FOR MODIFY (`create/update/delete`) 的 `failed/reported` 要按實體別名 (`-test`)，FOR DETERMINATION/FOR VALIDATION 的不用。

`%cid`：CREATE 情境下，被拒絕的這一系列還沒有真正的 Key (`id` 雖然是使用者指定的，但 RAP 框架的內部追蹤機制在 CREATE 尚未確認前一律用 `%cid` 識別)，所以 `failed-test / reported-test` 這裡用 `%cid = ls_create-%cid`，不是 `%key`——這跟 Part A 的 Validation (`on save`，資料已經存在，用 `%key`) 時機不同，用哪個要看「這筆資料當下有沒有正式 Key」。

✅ 驗證結果 (`programrun` 無頭執行，完全成功，還意外揭露一個 Managed/Unmanaged 行為差異)

```

MODIFY ENTITIES OF zi_rap03_umtest
ENTITY Test
  CREATE FIELDS ( id descr )
  WITH VALUE #(
    ( %cid = 'C1' id = 'VAL_TEST01' descr = 'Valid Row' )
    ( %cid = 'C2' id = 'VAL_TEST02' descr = '' ) )
  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).

```

一次 EML 呼叫同時傳一筆合法 (**VAL_TEST01**) 跟一筆不合法 (**VAL_TEST02** , **descr** 空字串) 的資料，輸出：

```

before EML
after EML, failed count: 1
reported message: Description must not be empty
after commit entities
rows found in DB: 1
VAL_TEST01 Valid Row

```

failed 正確只有 1 筆 (**VAL_TEST02**)，資料庫裡也正確只有 **VAL_TEST01** 這一筆合法資料——驗證成功。

⚠ 但這裡有一個值得注意的行為差異，呼應 **Part A** 提過的「**Managed Validation** 失敗會拒絕整個交易」：官方文件講的是 Managed——同一個 **RAP** 交易裡只要有一筆不合法，連同一批次裡的合法資料一起被拒絕。但這一課的 Unmanaged **create** 方法是逐列處理 (**LOOP AT it_create**)，不合法的那一列被 **CONTINUE** 跳過、合法的那一列照樣 **INSERT**——這次驗證結果剛好證實了這個差異：**VAL_TEST01** (合法) 依然成功寫進資料庫，即使同一次呼叫裡 **VAL_TEST02** (不合法) 被拒絕。這不是 bug，是 Unmanaged 的必然結果——**RAP** 框架不會替你的手寫邏輯強加「全部成功或全部失敗」這種語意，你的程式碼怎麼寫，行為就是怎樣；如果你想要 Unmanaged 也做到「一筆不合法、整批全部回滾」，要自己在 **create** 方法裡先完整檢查完所有列，只要有一筆不合法就整批都不 **INSERT** (或是用 Draft-enabled Unmanaged，那樣才能用回官方的宣告式 Validation，享有框架保證的整批拒絕語意，但這已經超出這門課的範圍)。

Eclipse ADT：在既有 BDEF 裡加一個 Validation——Step by Step

跟 rap05 加 Determination 的操作幾乎一樣 (沿用同一套 Eclipse 操作模式)：

1. Eclipse 打開目標 BDEF，在 { } 區塊裡手打 **validation <名稱> on save { field <欄位>; }** (沒有精靈能自動產生這一行)。
2. 確認 header 已經是 **managed implementation in class <類別名> unique;** (rap05 加 Determination 時應該已經改過，如果這是你第一次加自訂邏輯才需要改)。
3. 存檔 (**Ctrl+S**) + 啟用 (**Activate**)。
4. 在 Local Types Include 裡手動補上 **METHODS <名稱> FOR VALIDATION <alias>~<val名稱> IMPORTING keys FOR <alias>.** 宣告 + 對應的 **METHOD ... ENDMETHOD.** 實作 (跟 Determination 一樣，Eclipse 目前沒有針對「補一個 Validation」的專屬快速修正)。
5. 存檔 + 啟用整個類別。

練習：幫自己的 BDEF 加一個 Validation

輪到你了，接續 **rap05** 練習做的物件：

① 必做 (**Managed** , 純語法練習) : 對你 rap05 練習加了 Determination 的 Managed BDEF (`ZI_RAP02_FLIGHT_PRACTICE`) , 加一個 Validation——檢查某個欄位是否符合特定規則 (例如某個代碼欄位長度不能是空的、或值要在某個範圍內) , 練習 `validation ... on save { field ... }` 語法 + `FOR VALIDATION` Handler Method 怎麼寫。驗收依據 : `checkruns / sap_inactive_objects` 確認啟用成功即可。

② 選做 (**Unmanaged** , 進階挑戰 , 真的能跑) : 如果 rap05 練習 ② 有自己建 Unmanaged 物件 , 幫它的 `create` 方法加一個驗證規則 (可以是欄位不能為空、格式檢查、或範圍檢查) , 用 `failed-<alias>` / `reported-<alias>` 拒絕不合法的輸入 , 最後用 EML 驗證 : ①合法資料成功寫入 ②不合法資料被拒絕且 `failed` 正確回報。

完成後跟我說一下建立過程跟最終狀態 , 我會幫你核對。

Part C : Managed vs Unmanaged Validation 差異總表

	Managed	Unmanaged
宣告方式	BDEF 裡 <code>validation <名稱> on save { }</code> , 宣告式 , 只能 <code>on save</code> (沒有 <code>on modify</code>)	沒有宣告 , 邏輯直接寫在 <code>CREATE/UPDATE</code> 方法本體裡
失敗時的範圍	框架保證 : 一筆不合法 , 整個 RAP 交易 (Transactional Buffer) 裡的所有變更全部被拒絕	完全看你怎麼寫 : 這一課的逐列處理範例是「不合法的那筆被拒絕 , 合法的照樣寫入」 , 跟 Managed 行為不同
Handler Method 的 <code>failed/reported</code> 型別 (單一實體 BDEF)	直接是表格 , 不用 <code>-<alias></code>	直接是結構 , 要用 <code>-<alias></code> (<code>FOR MODIFY</code> Handler 的規則 , 跟 Determination / Validation Handler 不同)
這系統支援度	Draft 或非 Draft 都支援語法 (但這系統 CUD 執行不了)	官方文件明講 : 非 Draft 完全不支援 , 這門課沒教 Draft
這系統能不能真正執行	❌ (Managed Runtime 白名單限制)	✅ 已驗證成功
適合的情境	資料完整性規則 , 想要框架保證「要嘛全部成功、要嘛全部失敗」	需要更彈性的錯誤處理邏輯 (例如部分成功也可以接受) ; 或這系統這種 Managed CUD 執行不了的情況下的唯一選擇

學習目標

- 能寫出這系統適用的 Managed Validation 語法 : `validation <名稱> on save { field <欄位>; }` , 知道 Validation 只能 `on save` , 沒有 `on modify`
- 能寫出這系統要求的 obsolete Handler Method 語法 : `METHODS <名稱> FOR VALIDATION <alias>~<val名稱> IMPORTING keys FOR <alias>`
- 能講出 Validation 失敗時 , Managed 框架「整批拒絕」跟 Unmanaged 手寫邏輯「行為看你怎麼寫」的差異 , 並用這一課的驗證結果 (`VAL_TEST01` 照樣寫入) 具體說明

- 知道同一個 `obsolete Handler Method` 家族裡，`FOR MODIFY ( create/update/delete )` 的 `failed/reported` 要用 `-<alias>` 分類，`FOR DETERMINATION/FOR VALIDATION` 的（單一實體時）不用
- 能用 `new_message_with_text( severity = ... text = ... )` 建立一個最簡單的 RAP 錯誤訊息
- 能說出官方文件對 `Validation` 可用範圍的明確限制：跟 `Determination` 一樣，非 `Draft Unmanaged` 完全不支援
- 能在 `Unmanaged` 實作類別的 `CREATE` 方法裡設計「檢查失敗就 `APPEND` 到 `failed/reported` 並 `CONTINUE`」的驗證等效模式

物件清單

物件	名稱	型別	可執行性
Managed Behavior Definition (延伸，加 validation)	<code>ZI_RAP02_TASK</code>	<code>BDEF/BDO</code>	語法正確，CUD 無法執行
Managed 實作類別 (延伸，加 <code>validateStatus</code>)	<code>ZBP_I_RAP02_TASK</code>	<code>CLAS/OC</code>	語法正確，無法執行
Unmanaged 實作類別 (延伸， <code>CREATE</code> 加驗證邏輯)	<code>ZBP_I_RAP03_UM4</code>	<code>CLAS/OC</code>	✓
Unmanaged EML 驗證程式	<code>ZR_RAP06_VALDEMO</code>	<code>PROG/P</code>	✓ 已驗證成功

全部物件都在 `$TMP` 套件，`sap_inactive_objects` 確認 0 筆殘留。（這一課沒有新增 DDIC 物件，`ZRAP03_UMTEST / ZI_RAP03_UMTEST` 沿用 `rap05` 延伸過的版本，欄位沒有再變動。）

驗證方式

1. **Managed** 部分：`checkruns / sap_inactive_objects` 確認語法正確、成功啟用即可——不要嘗試執行，原因同 `rap03 Part C / rap05`
2. **Unmanaged** 部分：已用 `programrun` 完整驗證成功，一次 EML 呼叫同時傳合法與不合法資料，確認 `failed` 正確回報 1 筆、資料庫只有合法那筆寫入

思考題

1. 這一課的 `Unmanaged create` 方法，不合法的資料被拒絕後，合法的資料照樣寫入資料庫——如果你的業務需求是「只要這批裡有一筆不合法，全部都不要寫入」（模仿 `Managed` 的行為），程式碼要怎麼改？（提示：想想能不能分成兩個迴圈，先檢查完全部再決定要不要寫入）
2. `field(mandatory) description` 是 `rap03` 就有的欄位控制，這一課的 `validation validateStatus`（`Managed`）/`descr IS INITIAL` 檢查（`Unmanaged`）都是「檢查資料合不合法」。`field(mandatory)` 跟 `Validation` 兩者的差異在哪裡？什麼情況下該用 `field(mandatory)`、什麼情況下要升級成 `Validation`？
3. `rap05` 的 `Determination` 跟這一課的 `Validation`，`Handler Method` 都用了「`obsolete`」語法家族。如果你在正式產品程式碼裡看到警告訊息說某個語法即將棄用，除了「這系統剛好只支援舊語法」這個特殊情況外，一般專案面對「新語法功能更完整但舊系統不支援」的情境，可能會怎麼做技術決策？

答案

Managed : `zi_rap02_task.bdef.abap` (延伸版) 、 `zbp_i_rap02_task.clas.abap` 、
`zbp_i_rap02_task.clas.locals_imp.abap` (延伸版 , 含 Determination + Validation 兩個 Handler) 。

Unmanaged : `zbp_i_rap03_um4.clas.locals_imp.abap` (延伸版 , `create` 方法加驗證邏輯) 、
`zr_rap06_valdemo.prog.abap` (EML 驗證程式 , 已驗證執行成功) 。